CSSE374 Software Design: Project

JavaLinter

Contents

Overview 2

What is a linter, anyway? 2

Rules 3

1. Team size 3

2. Individual contribution requirement 3

2. Push your work every day 3

3. Checkin with your instructor every class day 3

4. Document all design decisions and sources 3

5. Rely solely on your own team to do the work 4

6. Make good choices about how to access the program files 4

7. Show how you use the design ideas from this class 4

Features 4

Examples of linter checks 4

Features besides linter checks 5

How your system should work 5

Do you need a GUI? 5

Design 5

Use of AI's 5

Where do I start? 6

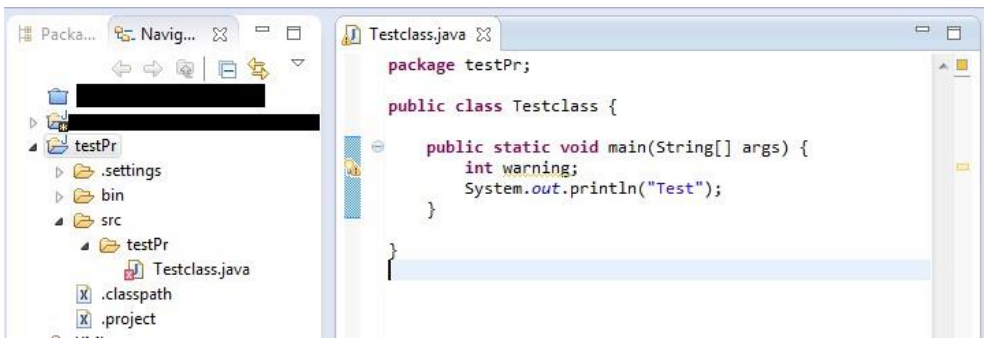
Grading – team project, individual grading 6

Overview

Your term project is to work with your assigned team to implement a Java *linter* to help your instructor grade CSSE374 assignments. Your instructor wants an easy way to detect known design anti-patterns in code, like "programming to the implementation rather than the interface." Your instructor also wants you to share ownership of the project vision and inject some creative checks of your own.

What is a linter, anyway?

Linters are basically the "spell checkers" of programming. Every IDE runs a built-in linter that underlines your code with those orange and red squiggles as you accidentally introduce bugs. Here is an example of the Eclipse linter catching an unused local variable:



The Eclipse linter detects an unused local variable. Source: https://stackoverflow.com/questions/18488813/eclipse-kepler-shows-error-marks-on-warnings

You can also run linters as stand-alone programs on the command line, like you see here:

A command-line linter catching two linting errors. Read more about linters: https://www.freecodecamp.org/news/what-is-linting-and-how-can-it-save-you-time/

Rules

1. Team size

This is a team project. Each team is composed of 4-5 people. For your team assignments, see "Team Assignments" in this Project Hub.

2. Individual contribution requirement

1) Everyone in the team must design and implement **one cursory style check, one principle violation check,** and **one pattern detector** (see Features section below and in-class discussion that are different from their team member's.

2) Everyone must contribute code under their name. Strive to write about the same amount of ***code*** as everyone else on your team. To track who does what, everyone ***must commit*** their own work with their own git login.

One may not use "pair programming" as a reason to explain the lack of code contribution for this project. We do encourage pair programming if your team finds it more efficient, but please make sure that every team member acts as the author evenly.

2. Push your work every day

Your instructor periodically will want to see how your project is going by looking at the team GitHub. Create your own team GitHub for this project. Give our TA and instructor access.

Incrementally push your work, at least once/day, so your instructor can see your progress.

3. Checkin with your instructor every class day

Show them your progress, the current design, and highlights of the current code. Discuss problems you are having, previous problems resolved. Listen to their critique of your design and show how you've fixed problems previously noted.

4. Document all design decisions and sources

Explain how you and your team decided all design decisions in your team's *engineering notebook.* **Everyone** needs to take their own notes on every design discussion they participate in and log those notes in this shared notebook. They may seem redundant now, but people can take away very different understandings of a design from a meeting.

If you found inspiration from someone/somewhere: **<u>cite</u>** it in your engineering notebook. It's important to your not-outright-failing-the-class.

Like what prompts you used to get what knowledge from an LLM, say.

5. Rely solely on your own team to do the work

At some point during the project, we will stop and ask ChatGPT to design the project, just to see what it can do. By point, you will have your own design you can compare this that with. This is the only time using AI tools will be legitimate in the design work for this project. Coding is a different thing! See the section "Use of AI's" *below*.

You also may collaborate with people outside your team in setting up the project. And use AI's for that. Like, "How do I import a Java Maven project into Visual Studio Code?

6. Make good choices about how to access the program files

You can read Java files for your Linter to analyze in two different ways. We provide ASM, a tool that reads Java ".class" object files, not the source files. You also can read ".java" source files directly to analyze things.

ASM sample code uses Maven to set up the embedded jar Any source of expertise to help you get that going is files.

7. Show how you use the design ideas from this class

Document how you use patterns and other design ideas we've talked about in CSSE374. In your final project video, each person should describe how they used those design methods to accomplish what they did in their features.

Features

This project is for your instructor's benefit to find problems with students' designs. **The precise list of features is open-ended.** We encourage you to research the features that you think go into a highquality linter, then write linter checks that would help find issues with a given design.

Examples of linter checks

As a guide, here is a big list of problems that the CSSE374 instructors imagined a design linter might catch. There are certainly more problems possible! Be creative.

- Poor names, everywhere!
- Violations of the CSSE374 design principles
- Classes that define one of the equals() and hashCode() methods, but not both methods
- Tight coupling
- Poor cohesion

- Redundant interfaces
- Instances of the "Bad Patterns"
- Poorly written implementations of good design patterns (Decorator, especially, can be tricky to get right)
- Missing implementations of abstract types
- Classes that can't be publicly constructed
- Global state

Features besides linter checks

A linter has many more considerations than just its checks. Like: what is the UI? What settings can you configure? How do you use the results? Where are Design Patterns used? Research some real linters and build a complete list of features before you begin.

How your system should work

Make it so that you can test individual features separately. For example, by testing your system on a couple of sample Java systems which do or don't pass one of the above checks or have one of the features. As you are building the system, it helps if you can just test the one thing working on. And, in the video at the end, you are needs to show clearly how the linter works with the features each person developed.

Most teams resort to having a file with lists of what projects to run as to exercise particular ones of the above features. Then, you add new features, you can add the projects that exercise those (along with possibly associated info like who did it and the feature that project goes with). When your system starts a run, it reads this file and asks the user what they want to do.

Do you need a GUI?

Some teams prefer to create a GUI for running their system. These can make it easy to pick, say, which design principles and features are exercised in a given run. But you also could do this from a command line in the console.

Design

This being a design course, we care how you design this project system! We want you to use as many of the following concepts from the course *as make sense* to design this system well. At various points during the development, we'll ask you to describe your design and name the concepts you are using. In the video presentation at the end, you are expected to show where you used these concepts:

1. Great software – What about the design achieves the functionality required, makes it flexible and maintainable.

2. How the system is designed in 3 layers with calls almost all going down.
3. How you used the patterns we studied which are advantageous in your system: a. Strategy
    b. Observer
    c. Adapter
    d. Façade
    e. Decorator
    f. Template
4. How you used these design principles:
    a. Open-Closed
    b. Hollywood
    c. Single Responsibility
    d. Program to Interface
    e. Principle of Least Knowledge
    f. Dependency Injection
5. How you maintained Coupling and Cohesion principles 6. How you followed Architecture principles:
    a. Identify architecture drivers
    b. Consider design concepts
        i. Reference Architectures ii.
        Architecture Patterns

Where do I start?

1. Create the team's repository on GitHub and send your instructor and TA invitations to join.
2. Read the **ASM User Guide** and explore the **example ASM code** on Moodle to learn how to write a program that can read Java code.
3. Do everything a software development professional would do to build Great Software (establish good team communication, understand the requirement, decompose the system into smaller modules, define test values first before you write the code, commitand push often etc.)

Grading – team project, individual grading

While the project is a team effort, we will mainly be grading*your work* as indicated by your engineering notebook, git blame, and git log. Demonstrate how *you*:

- Write Great Software.
- Document decisions in your engineering notebook.
- Apply test-driven development.
- Proactively manage your project.
- Be a good team member.

To earn a "4": in addition to the above, we expect you to further individually research cool design ideas that we did not talk about. Document what you learned in you engineering notebook. Experiment with candidate designs that include what you learned. Make new features that incorporate successful candidate designs. We should be able to see how you solved this design problem, reading your entry in the engineering notebook, and you should explain this in your team's final video.

An example of an individual extra feature requiring design work is this – "detecting excessive coupling in a system based on tunable rules" that you can pick as a Strategy, like how intensely classes interact to trigger reporting on them in the Linter, or alternative ways to judge closeness of class interactions.

Occasionally, we allow a team to do one large additional design activity together for this. An example would be "creating a web page front 3nd to run the testing and increase the usability of the system." It needs to be clear that every team member contributed to this larger feature's design.